プログラミングII 3Q-06
文字列とポインタ
(第11章)

第4回・第5回の課題解答例を解説します。

・WebClass: 成績→マイレポート

・3Q04フォルダおよび3Q05フォルダの確認

・教科書、筆記用具

勝田・田辺

課題の解説

解答例のソースコードは配布していません。
必要に応じてメモを取りましょう。

第4回・第5回の課題

4-2: 演習10-3（P271）3つの値のソート
4-3: ex03.c内のsqrt_set関数の実装
4-4: ex04.c内のsearch_array関数の実装

5-2: 演習11-4（P298）put_string関数の実装（添字演算子無し）
5-3: 演習11-5（P298）str_cnum関数の実装（添字演算子無し）
5-4: 演習11-6（P298）str_cnr関数の実装（添字演算子無し）

※constについての詳細は
https://programming-place.net/ppp/contents/c/032.html#const
等を参照のこと。

List 11-6 (P.294)

文字列のコピー

```c
#include <stdio.h>
int main(void)
{
	char str[128] = "ABC";
	char tmp[128];
	scanf("%s", tmp);
	str_copy(str, tmp);
	printf("str = \"%s\"\n", str);
	return 0;
}
```

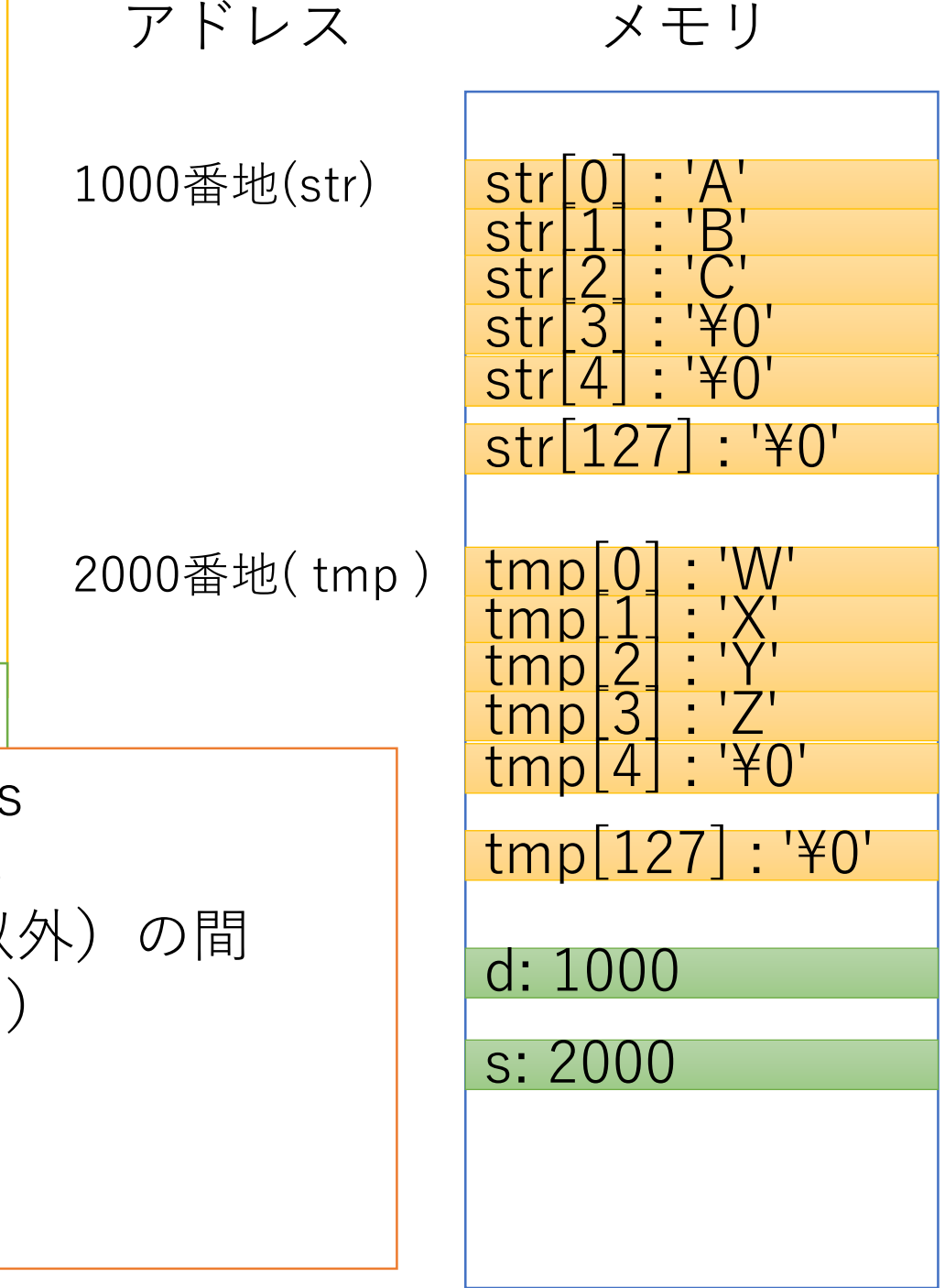
WXYZ

```c
void str_copy(char *d, const char *s)
{
	while(*d++ = *s++)
		;
}
```

- 代入*d = *s
- 式の値は*s
  (非0('¥0'以外) の間
  繰り返す)
- 代入後、

d++

s++



```c
#include <stdio.h>
int main(void)
{
	char str[128] = "ABC";
	char tmp[128];
	scanf("%s", tmp);
	str_copy(str, tmp);
	printf("str = \"%s\"\n", str);
	return 0;
}
```

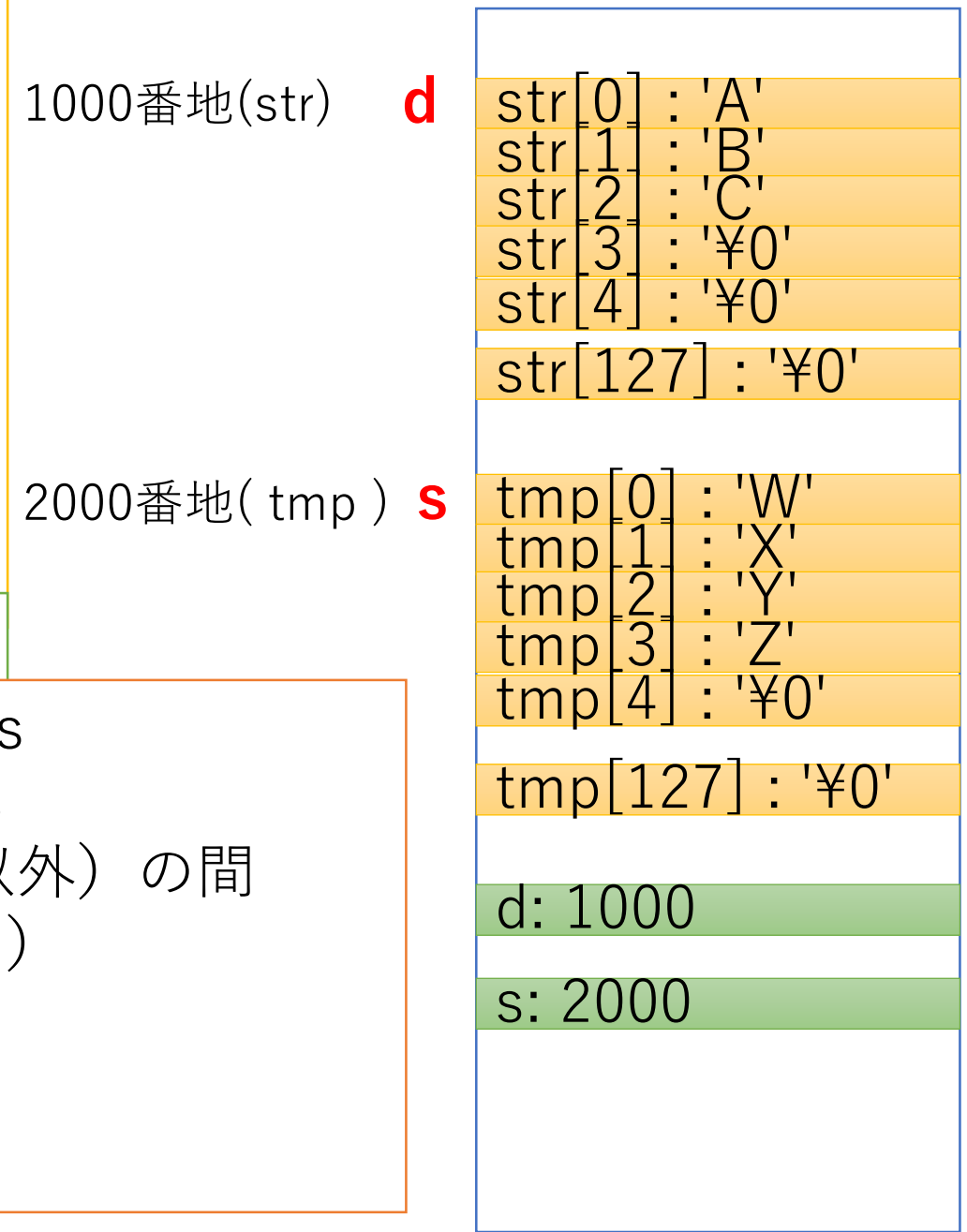
WXYZ

```c
void str_copy(char *d, const char *s)
{
	while(*d++ = *s++)
		;
}
```

- 代入*d = *s
- 式の値は*s
  (非0('¥0'以外) の間
  繰り返す)
- 代入後、

d++

s++



```c
#include <stdio.h>
int main(void)
{
	char str[128] = "ABC";
	char tmp[128];
	scanf("%s", tmp);
	str_copy(str, tmp);
	printf("str = \"%s\"\n", str);
	return 0;
}
```

WXYZ

```c
void str_copy(char *d, const char *s)
{
	while(*d++ = *s++)
		;
}
```

- 代入*d = *s
- 式の値は*s
  (非0('¥0'以外) の間
  繰り返す)
- 代入後、

d++

s++

アドレス　　メモリ

1000番地(str)　d

str[0] : 'W'
str[1] : 'B'
str[2] : 'C'
str[3] : '¥0'
str[4] : '¥0'
str[127] : '¥0'

2000番地(tmp)　s

tmp[0] : 'W'
tmp[1] : 'X'
tmp[2] : 'Y'
tmp[3] : 'Z'
tmp[4] : '¥0'
tmp[127] : '¥0'

d: 1001

s: 2001

```c
#include <stdio.h>
int main(void)
{
	char str[128] = "ABC";
	char tmp[128];
	scanf("%s", tmp);
	str_copy(str, tmp);
	printf("str = \"%s\"\n", str);
	return 0;
}
```

WXYZ

```c
void str_copy(char *d, const char *s)
{
	while(*d++ = *s++)
		;
}
```

- 代入*d = *s
- 式の値は*s
  (非0('¥0'以外) の間
  繰り返す)
- 代入後、

d++

s++

アドレス　　メモリ

1000番地(str)

d

str[0] : 'w'
str[1] : 'X'
str[2] : 'C'
str[3] : '¥0'
str[4] : '¥0'

str[127] : '¥0'

2000番地(tmp)

s

tmp[0] : 'W'
tmp[1] : 'X'
tmp[2] : 'Y'
tmp[3] : 'Z'
tmp[4] : '¥0'

tmp[127] : '¥0'

d: 1002

s: 2002

```c
#include <stdio.h>
int main(void)
{
	char str[128] = "ABC";
	char tmp[128];
	scanf("%s", tmp);
	str_copy(str, tmp);
	printf("str = \"%s\"\n", str);
	return 0;
}
```

WXYZ

```c
void str_copy(char *d, const char *s)
{
	while(*d++ = *s++)
		;
}
```

- 代入*d = *s
- 式の値は*s
  (非0('¥0'以外) の間
  繰り返す)
- 代入後、

d++

s++

アドレス　　メモリ

1000番地(str)

d

str[0] : 'W'
str[1] : 'X'
str[2] : 'C'
str[3] : '¥0'
str[4] : '¥0'
str[127] : '¥0'

2000番地(tmp)

s

tmp[0] : 'W'
tmp[1] : 'X'
tmp[2] : 'Y'
tmp[3] : 'Z'
tmp[4] : '¥0'
tmp[127] : '¥0'

d: 1003

s: 2003

```c
#include <stdio.h>
int main(void)
{
	char str[128] = "ABC";
	char tmp[128];
	scanf("%s", tmp);
	str_copy(str, tmp);
	printf("str = \"%s\"\n", str);
	return 0;
}
```

WXYZ

```c
void str_copy(char *d, const char *s)
{
	while(*d++ = *s++)
		;
}
```

- 代入*d = *s
- 式の値は*s
  (非0('¥0'以外) の間
  繰り返す)
- 代入後、

d++

s++

アドレス　　メモリ

1000番地(str)

str[0] : 'W'
str[1] : 'X'
str[2] : 'C'
str[3] : 'Z'
str[4] : '¥0'

d

str[127] : '¥0'

2000番地(tmp)

tmp[0] : 'W'
tmp[1] : 'X'
tmp[2] : 'Y'
tmp[3] : 'Z'
tmp[4] : '¥0'

s

tmp[127] : '¥0'

d: 1004

s: 2004

```c
#include <stdio.h>
int main(void)
{
	char str[128] = "ABC";
	char tmp[128];
	scanf("%s", tmp);
	str_copy(str, tmp);
	printf("str = \"%s\"\n", str);
	return 0;
}
```

WXYZ

```c
void str_copy(char *d, const char *s)
{
	while(*d++ = *s++)
		;
}
```

- 代入*d = *s
- 式の値は*s
  (非0('¥0'以外) の間
  繰り返す)
- 代入後、

d++

s++

アドレス　　メモリ

1000番地(str)

d

str[0] : 'W'
str[1] : 'X'
str[2] : 'C'
str[3] : 'Z'
str[4] : '¥0'
str[127] : '¥0'

2000番地(tmp)

s

tmp[0] : 'W'
tmp[1] : 'X'
tmp[2] : 'Y'
tmp[3] : 'Z'
tmp[4] : '¥0'
tmp[127] : '¥0'

d: 1005

s: 2005

```c
#include <stdio.h>
int main(void)
{
        char str[128] = "ABC";
        char tmp[128];
        scanf("%s", tmp);
        str_copy(str, tmp);
        printf("%s\n", str);
        printf("%s\n", str_copy(str, tmp));
}
```

WXYZ

1000番地

```c
char * str_copy(char *d, const char *s)
{
        char *t = d;
        while(*d++ = *s++)
                ;
        return t;
}
```

1000番地

アドレス　　メモリ

1000番地(str)

d

str[0] : 'W'
str[1] : 'X'
str[2] : 'C'
str[3] : 'Z'
str[4] : '¥0'
str[127] : '¥0'

2000番地(tmp)

s

tmp[0] : 'W'
tmp[1] : 'X'
tmp[2] : 'Y'
tmp[3] : 'Z'
tmp[4] : '¥0'
tmp[127] : '¥0'

d: 1005

s: 2005

t: 1000

List11-6の修正

- printf("コピーするのは：", tmp); //第2引数不要
- while ((*d++ = *s++)) //-Wparantheses警告の対策

　if(a = b) とすると、「a == b」の間違いでは？とコンパイラが判断し、「警告: 真偽値として使われる代入のまわりでは、丸括弧の使用をお勧めします [-Wparentheses]」的なメッセージを出す。

　if((a = b)) として、「あえて代入式を使っています」とアピールすると警告が消える。

List 11-7 (P.296)

文字列のコピー（誤った例）

```c
#include <stdio.h>
int main(void)
{
	char *ptr = "ABC";
	char tmp[128];
	scanf("%s", tmp);
	str_copy(ptr, tmp);
	printf("ptr = \"%s\"\n", ptr);
	return 0;
}
```

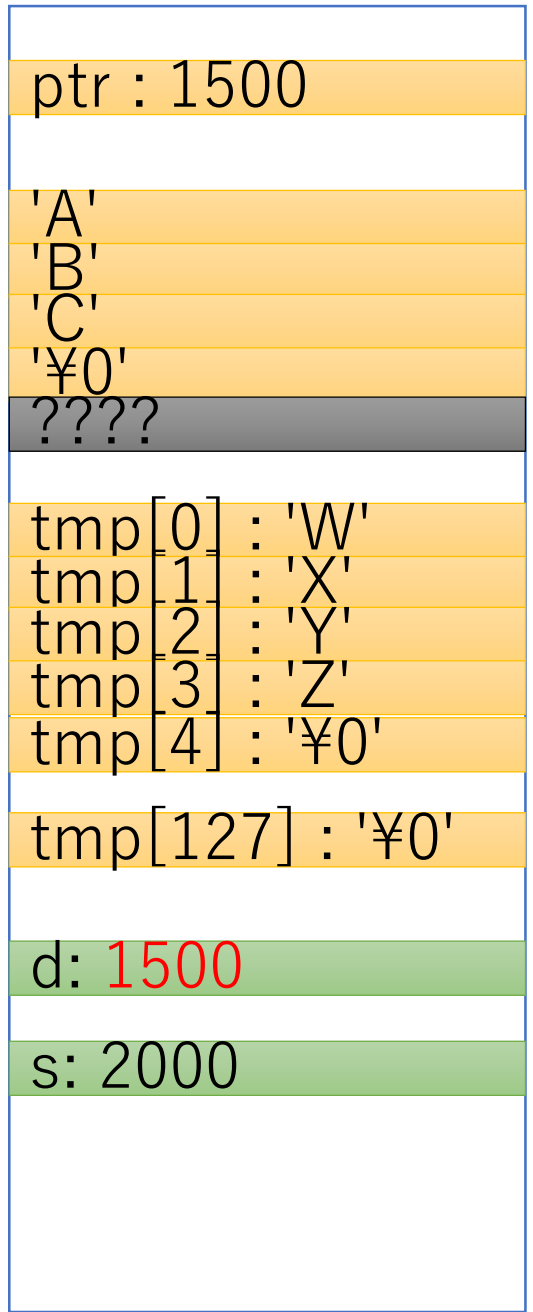
WXYZ

```c
void str_copy(char *d, const char *s)
{
	while(*d++ = *s++)
		;
}
```

- 代入*d = *s
- 式の値は*s
  (非0('¥0'以外) の間
  繰り返す)
- 代入後、

d++

s++



```c
#include <stdio.h>
int main(void)
{
	char *ptr = "ABC";
	char tmp[128];
	scanf("%s", tmp);
	str_copy(ptr, tmp);
	printf("ptr = \"%s\"\n", ptr);
	return 0;
}
```

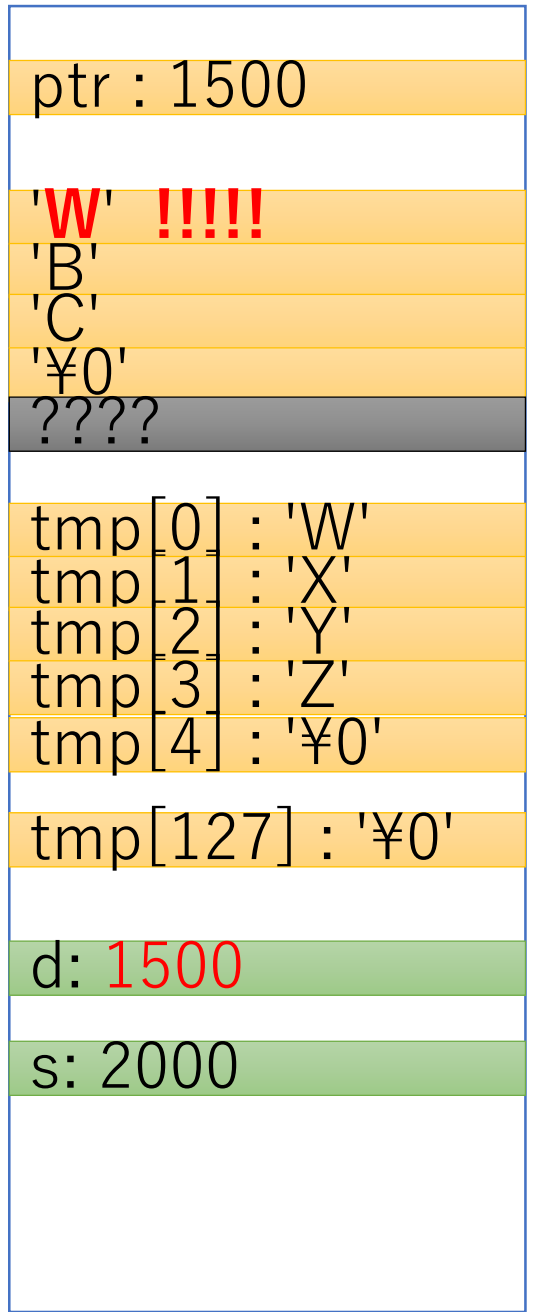
WXYZ

```c
void str_copy(char *d, const char *s)
{
	while(*d++ = *s++)
		;
}
```

- 代入*d = *s
- 式の値は*s
  (非0('¥0'以外) の間
  繰り返す)
- 代入後、

d++

s++



```c
#include <stdio.h>
int main(void)
{
	char *ptr = "ABC";
	char tmp[128];
	scanf("%s", tmp);
	str_copy(ptr, tmp);
	printf("ptr = \"%s\"\n", ptr);
	return 0;
}
```

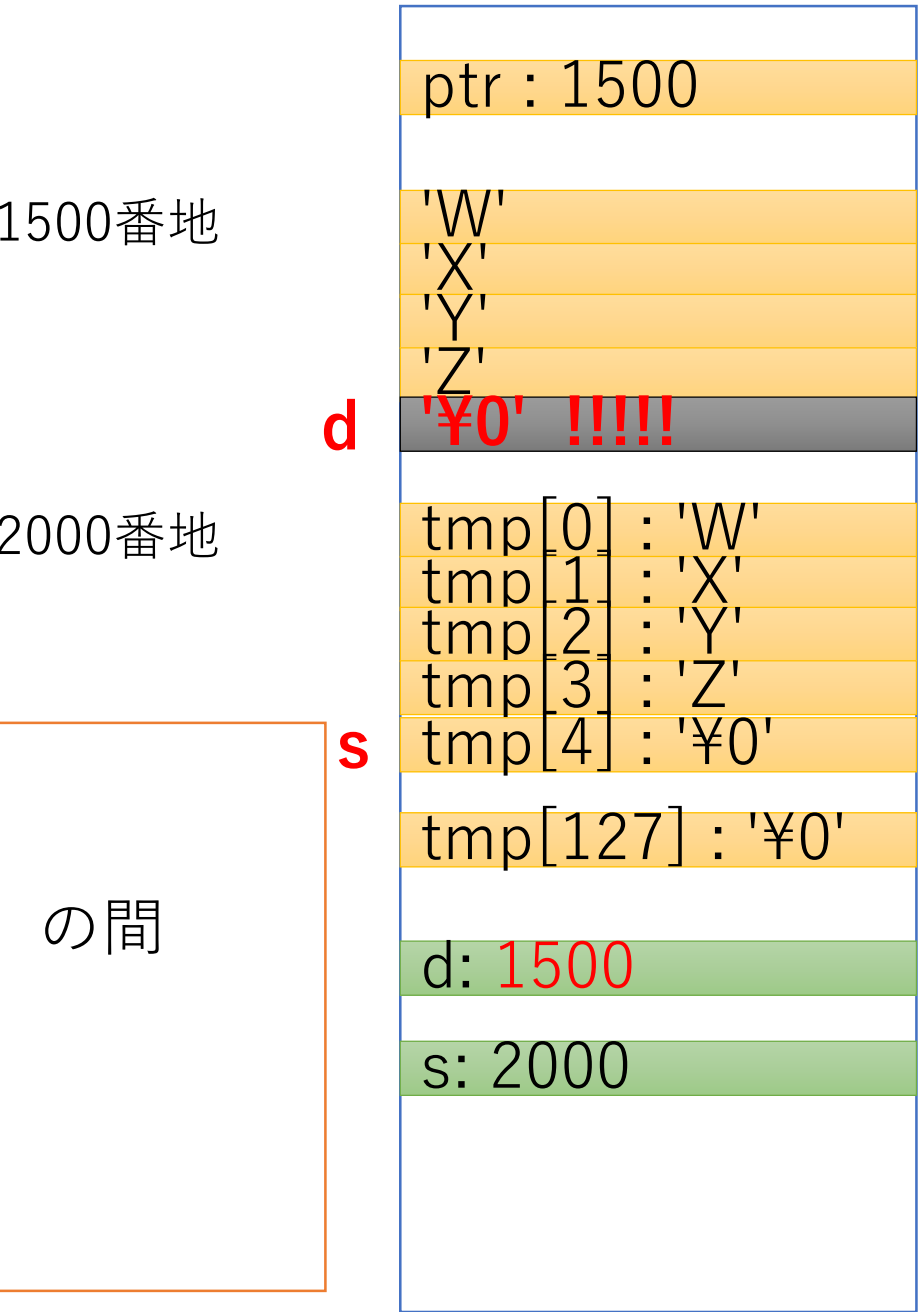
WXYZ

```c
void str_copy(char *d, const char *s)
{
	while(*d++ = *s++)
		;
}
```

- 代入*d = *s
- 式の値は*s
  (非0('¥0'以外) の間
  繰り返す)
- 代入後、

d++

s++



```c
#include <stdio.h>
int main(void)
{
	char *ptr = "ABC";
	char tmp[128];
	scanf("%s", tmp);
	str_copy(ptr, tmp);
	printf("ptr = \"%s\"\n", ptr);
	return 0;
}
```

WXYZ

```c
void str_copy(char *d, const char *s)
{
	while(*
		;
}
```

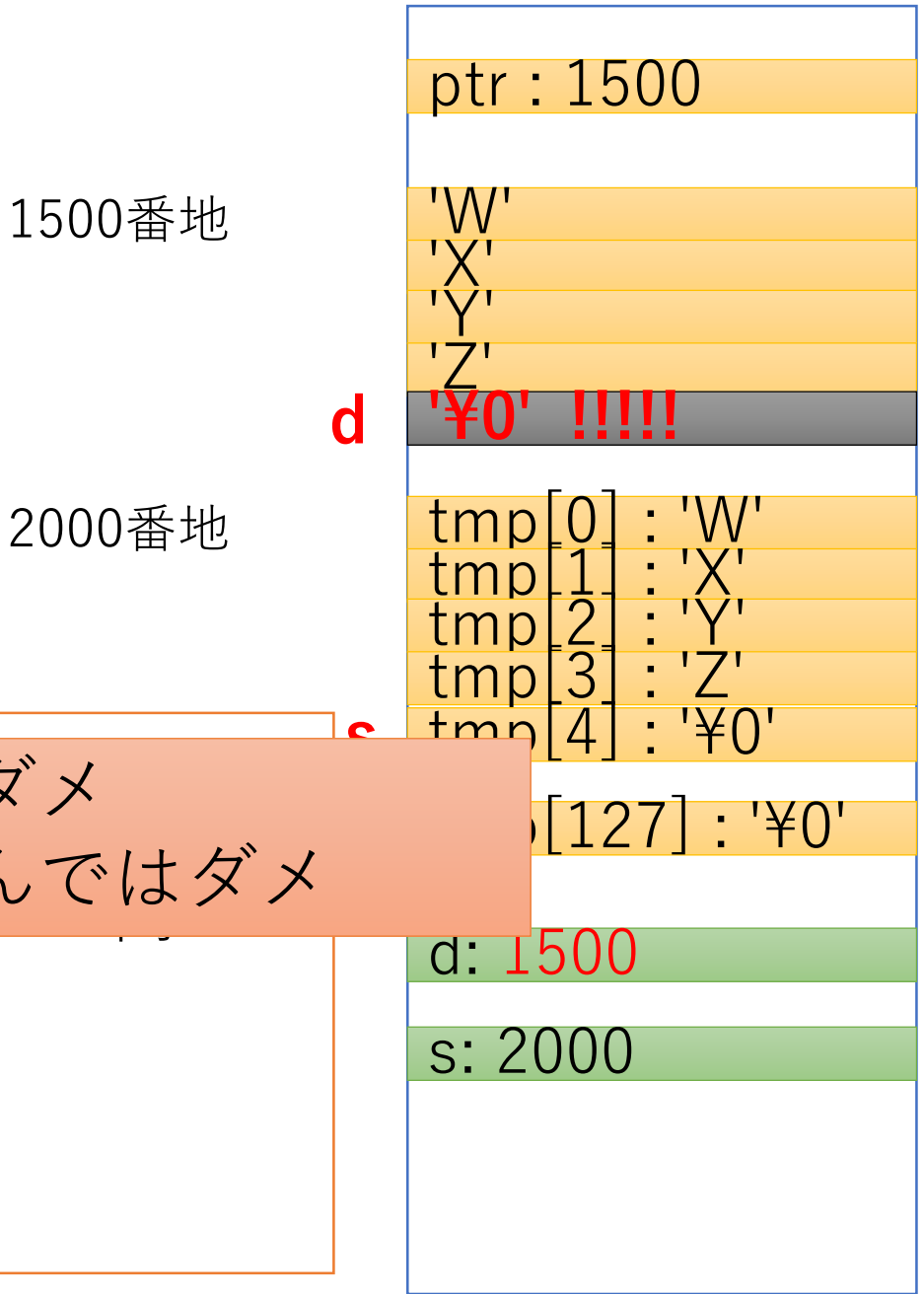
文字列リテラルを書き換えてはダメ
確保されていない領域に書き込んではダメ

繰り返す）

- 代入後、

d++

s++



11-3章

文字列を扱うライブラリ関数

・使えるようになろう！

・余裕のある人は、実装例を見て理解を！

strlen使用例 (sample01.c)

```
#include<stdio.h>
#include<string.h> //必要
int main(void){
	char *p = "ABCDE";
	char str[10] = "FGH";
	//%luは「long unsigned int」の書式指定。
	printf("%lu¥n", strlen(p)); //5
	printf("%lu¥n", strlen(str));//3
	return 0;
}
```

strcmp使用例 (sample02.c)

```
#include<stdio.h>
#include<string.h>
int main(void){
  char ans[20] = "programming"; //正解
  char input[20]; //ユーザ入力
  printf("「プログラミング」の綴りを英語で書いてください");
  「※」
  printf("終了します。¥n");
  return 0;
}
```

```
  「※」部分
  while(1){
    scanf("%s", input); //入力する
    if(strcmp(input, ans) == 0){ //一致
      printf("正解です！¥n");
      break; //whileを抜け出す
    }
    else { //不正解
       printf("残念！再度入力して：");
    }
  }
```

atoi使用例 (List11-12 / sample03.c)

```
#include <stdio.h>
#include <stdlib.h> //必要
int main(void)
{
	char str[128];
	printf("文字列を入力してください：");
	scanf("%s", str);
	printf("整数に変換すると%dです。¥n", atoi(str));
	return 0;
}
```

休憩

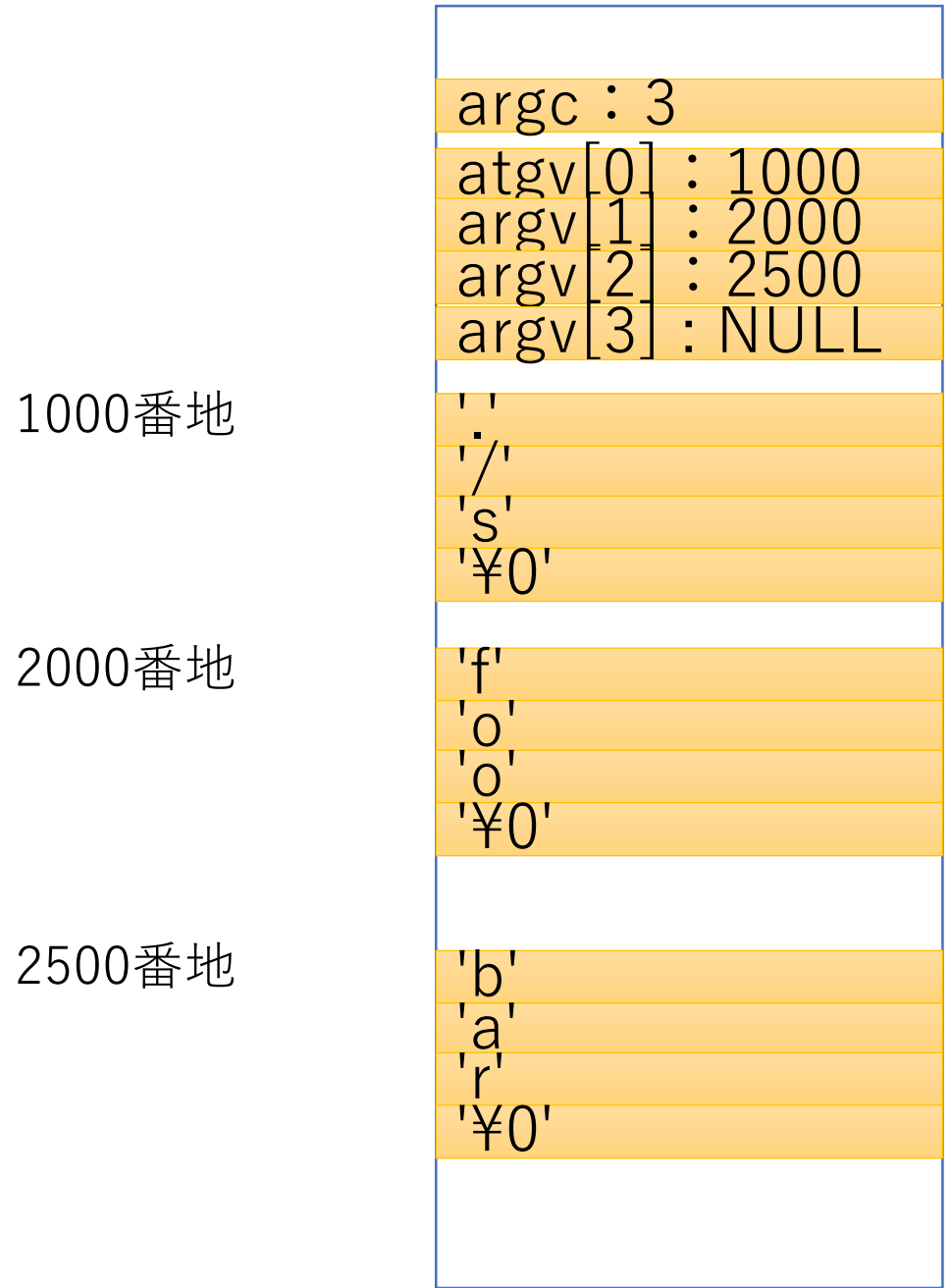
main関数

Programming Place Plus「C言語編 言語学習 第45章「コマンドライン」」
https://programming-place.net/ppp/contents/c/045.html

```c
/* sample04.c */
#include <stdio.h>
int main(int argc, char* argv[])
{
    for( int i = 0; i < argc; ++i ){
        puts( argv[i] );
    }
    return 0;
}
```

```
$ gcc -Wall -o s.exe sample04.c

tanabe@S-06-2002 ~/svn/cslab/tan
$ ./s.exe foo bar
./s
foo
bar
```



課題1

課題: 引数として整数を受け取り、その整数の合計を表示するプログラムを作成せよ。

実行例:

$ ./ex01.exe 100 200 300

合計は600です。

条件

・整数の個数は3個とは限らない。

・すべての引数が整数であると仮定して良い。

・argv[0]はプログラム名なので加算の対象としないこと。

課題2

課題: 最初に「-avg」オプションが指定された場合、合計ではなく平均値を表示するようにプログラムを変更せよ。

実行例1:

$ ./ex02.exe 100 200 -50

合計は250です。

実行例2:

$ ./ex01.exe –avg 100 200 -50

平均は83.333333です。